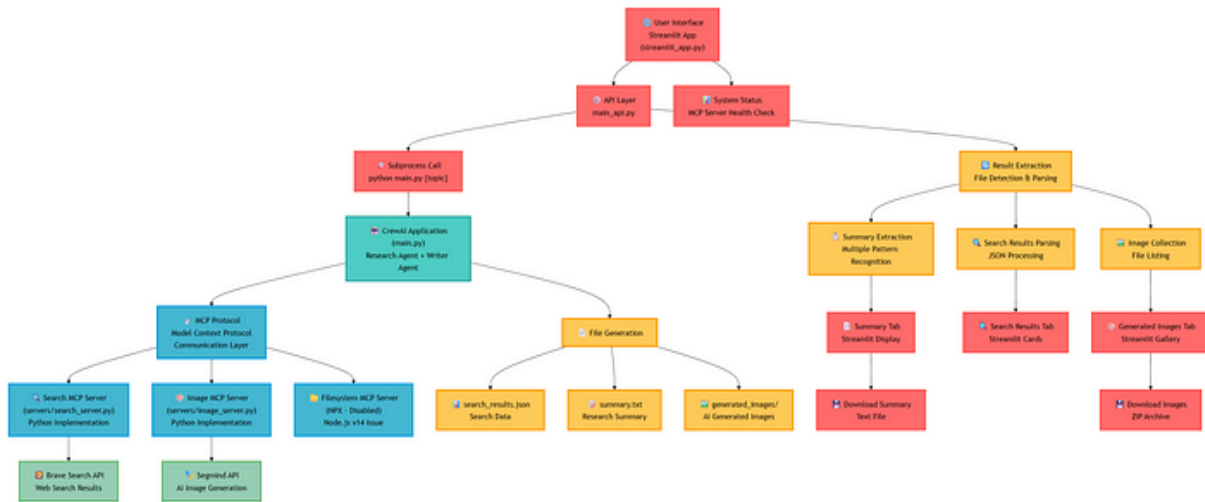


Coding Guide

🚀 Building an AI-Powered Study Assistant with MCP, CrewAI, and Streamlit 🔬



🌟 Introduction: The Future of AI-Powered Research is Here

Imagine having a personal research assistant that can instantly search the web, analyze information, generate comprehensive summaries, and even create relevant images — all through a beautiful web interface. What if this assistant could be powered by cutting-edge AI agents that collaborate seamlessly with external tools and APIs?

Meet the MCP-Powered Study Assistant — a revolutionary application that combines the best of modern AI technologies:

- 🤖 **CrewAI agents** for intelligent research and writing
- 🦊 **Model Context Protocol (MCP)** for seamless tool integration
- 🌐 **Streamlit** for an intuitive web interface
- 🔍 **Real-time web search** via Brave Search API
- 🎨 **AI image generation** through Segmind API

This isn't just another ChatGPT wrapper — it's a complete research ecosystem that demonstrates the future of AI-powered applications. In this comprehensive guide, we'll walk through every implementation detail, from MCP server creation to beautiful UI design.

🔧 Technology Stack: The Building Blocks of Innovation

Core Framework

- 🤖 **CrewAI**: Multi-agent AI framework for orchestrating intelligent workflows
- 🦊 **MCP (Model Context Protocol)**: Standardized protocol for AI-tool integration
- 🌐 **Streamlit**: Modern web framework for AI applications
- 🐍 **Python 3.8+**: Primary programming language

External APIs & Services

- 🦁 **Brave Search API**: Real-time web search capabilities
- 🎨 **Segmind API**: State-of-the-art AI image generation
- 🧠 **Groq -llama-3.3-70b-versatile**: Large language model for intelligent processing

Development Tools

- 📦 **JSON**: Data interchange format
- 🛠️ **Subprocess**: Process management for modular architecture
- 📁 **File System**: Result storage and retrieval
- 🎨 **CSS**: Custom styling for enhanced UX

Server Type	Status	Dependency	Tools Available
Python Image Server	✅ Working	Python only	image_creation_segmind
Python Search Server	✅ Working	Python only	brave_search , search_news
NPX Filesystem Server	⚠️ Node.js v14	Node.js 18+ needed	File operations

🏗️ Architecture Overview: How It All Connects

Our Study Assistant follows a sophisticated multi-layered architecture:

 Streamlit UI →  API Layer →  CrewAI Agents →  MCP Protocol → 
MCP Servers →  External APIs

The Flow Breakdown:

1. **User Input:** Topic entered via Streamlit interface
2. **Process Orchestration:** API layer manages the research workflow
3. **AI Agent Activation:** CrewAI agents begin collaborative research
4. **Tool Integration:** MCP protocol connects agents with specialized servers
5. **Data Collection:** Search and image servers gather relevant content
6. **Result Processing:** Files generated and parsed for display
7. **User Experience:** Beautiful tabbed interface presents all results

Implementation Deep Dive: Building Each Component

1. Setting Up CrewAI Agents

The heart of our system lies in two specialized AI agents:

 **Research Agent:** Web search specialist

```
researcher = Agent(  
    role='Research Specialist',  
    goal='Conduct comprehensive research on {topic}',  
    backstory='Expert at finding and analyzing information',  
    tools=[search_tool],  
    verbose=True  
)
```

 **Writer Agent:** Content synthesis expert

```
writer = Agent(  
    role='Content Writer',  
    goal='Create comprehensive study materials',  
    backstory='Skilled at organizing complex information',  
    tools=[image_tool],  
    verbose=True  
)
```

2. Building MCP Servers

Search Server (servers/search_server.py)

```

@server.call_tool()
async def search_web(arguments: dict) -> list[TextContent]:
    """Brave Search API integration"""
    query = arguments.get("query", "")

    headers = {"X-Subscription-Token": BRAVE_API_KEY}
    params = {"q": query, "count": 10}

    response = requests.get(BRAVE_SEARCH_URL, headers=headers, params=params)
    results = response.json()

    return [TextContent(type="text", text=json.dumps(results))]

```

Image Server (servers/image_server.py)

```

@server.call_tool()
async def generate_image(arguments: dict) -> list[TextContent]:
    """Segmind API image generation"""
    prompt = arguments.get("prompt", "")

    data = {
        "prompt": prompt,
        "style": "photographic",
        "samples": 1
    }

    response = requests.post(SEGMIND_URL, json=data, headers=headers)
    # Save and return image path

```

3. Creating the Streamlit Interface

Beautiful UI with Custom Styling

```

def apply_custom_css():
    st.markdown("""
<style>
.main-header {
background: linear-gradient(90deg, #667eea 0%, #764ba2 100%);
padding: 2rem;
border-radius: 10px;
color: white;
text-align: center;
margin-bottom: 2rem;
}

.result-card {
background: white;
padding: 1.5rem;
border-radius: 10px;
box-shadow: 0 2px 4px rgba(0,0,0,0.1);
margin: 1rem 0;
}
</style>
""", unsafe_allow_html=True)

```

Multi-Tab Result Display

```
def display_results():
    tab1, tab2, tab3 = st.tabs([
        "🔍 Search Results",
        "📄 Summary",
        "🖼️ Generated Images"
    ])

    with tab1:
        display_search_results()
    with tab2:
        display_summary_with_download()
    with tab3:
        display_image_gallery()
```

4. API Layer Implementation

The `main_api.py` serves as a crucial bridge:

```
def run_research(topic: str) -> Dict:
    """Execute research workflow"""
    try:
        # Run main.py as subprocess
        result = subprocess.run(
            [sys.executable, "main.py", topic],
            capture_output=True,
            text=True,
            timeout=300 # 5-minute timeout
        )

        return {
            "search_results": extract_search_results(),
            "summary": extract_summary_from_output(result.stdout),
            "images": get_generated_images(),
            "success": True
        }
    except subprocess.TimeoutExpired:
        return {"success": False, "error": "Research timeout"}
```

Key Features: What Makes It Special

Intelligent Web Search

- Real-time Brave Search API integration
- Structured result parsing and filtering
- Relevance-based content ranking

Multi-Agent Collaboration

- Specialized research and writing agents
- Automated workflow orchestration
- Context-aware information synthesis

AI Image Generation

- Topic-relevant visual content creation
- Multiple image style options
- Automatic prompt optimization

Beautiful Web Interface

- Responsive design with custom CSS
- Tabbed result organization
- Download capabilities for all content

System Monitoring

- Real-time MCP server status
- Error handling and recovery
- Performance metrics tracking

Advanced Implementation Tips

1. Error Handling Strategy

```
def robust_mcp_call(server_path: str, max_retries: int = 3):
    for attempt in range(max_retries):
        try:
            # MCP server communication
            return call_mcp_server(server_path)
        except Exception as e:
            if attempt == max_retries - 1:
                st.error(f"🚨 Server unavailable: {e}")
            time.sleep(2 ** attempt) # Exponential backoff
```

2. Result Extraction Patterns

```
def extract_summary_from_output(output: str) -> str:
    patterns = [
        r"FINAL RESULT:\s*(.+?)(?=\n\n|\Z)",
        r"## Final Answer:\s*(.+?)(?=\n\n|\Z)",
        r"Summary:\s*(.+?)(?=\n\n|\Z)"
    ]

    for pattern in patterns:
        match = re.search(pattern, output, re.DOTALL | re.IGNORECASE)
        if match:
            return clean_summary_text(match.group(1))

    return "Summary extraction failed"
```

3. Performance Optimization

- Asynchronous MCP server calls
- Parallel file processing
- Intelligent caching strategies
- Resource cleanup automation

Deployment Guide: Getting Your Assistant Live

1. Environment Setup

```
pip install -r requirements.txt # Configure API keys export
BRAVE_API_KEY="your-brave-key" export SEGMIND_API_KEY="your-segmind-
key" export OPENAI_API_KEY="your-openai-key"
```

2. Launch Sequence

```
# Start MCP servers
python servers/search_server.py &
python servers/image_server.py &

# Launch Streamlit app
streamlit run streamlit_app.py
```

3. Production Considerations

- Container deployment with Docker
- Load balancing for high traffic
- Database integration for result persistence

API rate limiting and monitoring

4. Complete Code Implementation

📁 crewai_mcp/

- |— 📄 main.py # 🤖 Core CrewAI application with agents
- |— 📄 requirements.txt # 📦 Python dependencies
- |— 📄 debug_summary.py # 🔧 Summary extraction debugging tool
- |— 📄 app.py # 🌐 Beautiful web interface
- |— 📄 setup_nodejs.py # ⚙️ Node.js setup utility
- |— 📄 test_python_version.py # 🧪 Python version compatibility test
- |— 📄 segmin.py # 🎨 Segmind API utility
- |
- |— 📁 servers/ # 🚀 MCP Server Implementations
 - | |— 📄 search_server.py # 🔍 Brave Search MCP Server (Python)
 - | |— 📄 image_server.py # 🎨 Segmind Image MCP Server (Python)
 - | |
 - | |— 📁 search_results/ # 📊 Generated Search Data
 - | | |— ... (research topics)
 - | |
 - | |— 📁 images/ # 🖼️ Generated AI Images
 - | | |— ... (topic images)
- |
- |— 📁 __pycache__/ # 🐍 Python cache files
- |— 📁 .venv/ # 🔒 Virtual environment

🏠 Architecture Breakdown

📱 Frontend Layer

- streamlit_app.py — Main web interface with beautiful UI
- streamlit_app_backup.py — Backup version for safety

API & Integration Layer

- main_api.py — Bridge between Streamlit and CrewAI
- app.py — Alternative interface implementation

AI Core Layer

- main.py — CrewAI agents (Research + Writer)
- debug_summary.py — Summary extraction utilities

MCP Server Layer

- servers/search_server.py — Web search via Brave API
- servers/image_server.py — Image generation via Segmind API

Data Storage Layer

- servers/search_results/ — JSON files with search data (40+ topics)
- servers/images/ — Generated AI images (30+ visuals)

Configuration & Utilities

- requirements.txt — Dependencies management
- setup_nodejs.py — Environment setup
- test_python_version.py — Compatibility testing

MCP Server- Image_Server

```

    from typing import Any
import httpx
from mcp.server.fastmcp import FastMCP
import os
import requests
import base64
import logging
from pathlib import Path
from dotenv import load_dotenv
load_dotenv()

# Setup logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("image_server")

# Initialize FastMCP server
mcp = FastMCP("image_server")

# Get current directory
current_dir = Path(__file__).parent
output_dir = current_dir / "images"
os.makedirs(output_dir, exist_ok=True)

# Validate API key
api_key = os.getenv("SEGMIND_API_KEY")
if not api_key:
    logger.error("SEGMIND_API_KEY environment variable is not set!")
    raise RuntimeError("Missing Segmind API key")

url = "https://api.segmind.com/v1/imagen-4"

@mcp.tool(name="image_creation_openai", description="Create an image using Segmind API")
def image_creation_openai(query: str, image_name: str) -> str:
    try:
        logger.info(f"Creating image for query: {query}")

        # Request payload
        data = {
            "prompt": f"Generate an image: {query}",
            "negative_prompt": "blurry, pixelated",
            "aspect_ratio": "4:3"
        }

        headers = {'x-api-key': os.getenv("SEGMIND_API_KEY")}

        # Add timeout and error handling
        try:
            response = requests.post(url, json=data, headers=headers,
timeout=30)
            response.raise_for_status()
        except requests.exceptions.RequestException as e:
            logger.error(f"API request failed: {e}")
            return {"success": False, "error": f"API request failed: {str(e)}"}
    
```

```

        # Save the image
        image_path = output_dir / f"{image_name}.jpeg"
        with open(image_path, "wb") as f:
            f.write(response.content)

        logger.info(f"Image saved to {image_path}")
        return {"success": True, "image_path": str(image_path)}
    except Exception as e:
        logger.exception("Image creation failed")
        return {"success": False, "error": str(e)}

if __name__ == "__main__":
    logger.info("Starting Image Creation MCP Server")
    try:
        mcp.run(transport="stdio")
    except Exception as e:
        logger.exception("Server crashed")
        # Add pause to see error in Windows
        input("Press Enter to exit...")
        raise

```

MCP Server- Search_Server

```

from typing import Any, Dict, List
import requests
from mcp.server.fastmcp import FastMCP
import os
import logging
import json
from pathlib import Path
from dotenv import load_dotenv

load_dotenv()

# Setup logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("search_server")

# Initialize FastMCP server
mcp = FastMCP("search_server")

# Get current directory
current_dir = Path(__file__).parent
results_dir = current_dir / "search_results"
os.makedirs(results_dir, exist_ok=True)

# Validate API key
api_key = os.getenv("BRAVE_API_KEY")
if not api_key:
    logger.warning("BRAVE_API_KEY environment variable is not set!")
    logger.warning("Search functionality will be limited or unavailable")

# Brave Search API endpoint
BRAVE_SEARCH_URL = "https://api.search.brave.com/res/v1/web/search"

@mcp.tool(name="brave_search", description="Search the web using Brave Search API")
def brave_search(query: str, count: int = 10) -> Dict[str, Any]:
    """
    Search the web using Brave Search API

    Args:
        query: Search query string
        count: Number of results to return (max 20)

    Returns:
        Dictionary containing search results
    """
    try:
        logger.info(f"Searching for: {query}")

        if not api_key:
            return {
                "success": False,
                "error": "BRAVE_API_KEY not configured",
                "results": []
            }

        # Limit count to reasonable range

```

```

count = max(1, min(count, 20))

# Request headers
headers = {
    "Accept": "application/json",
    "Accept-Encoding": "gzip",
    "X-Subscription-Token": api_key
}

# Request parameters
params = {
    "q": query,
    "count": count,
    "search_lang": "en",
    "country": "US",
    "safesearch": "moderate",
    "freshness": "pw", # Past week for more recent results
    "text_decorations": False,
    "spellcheck": True
}

# Make API request
try:
    response = requests.get(
        BRAVE_SEARCH_URL,
        headers=headers,
        params=params,
        timeout=30
    )
    response.raise_for_status()
except requests.exceptions.RequestException as e:
    logger.error(f"Search API request failed: {e}")
    return {
        "success": False,
        "error": f"Search API request failed: {str(e)}",
        "results": []
    }

# Parse response
try:
    data = response.json()
except json.JSONDecodeError as e:
    logger.error(f"Failed to parse search response: {e}")
    return {
        "success": False,
        "error": "Failed to parse search response",
        "results": []
    }

# Extract and format results
search_results = []
web_results = data.get("web", {}).get("results", [])

for result in web_results:
    search_result = {
        "title": result.get("title", ""),

```

```

        "url": result.get("url", ""),
        "description": result.get("description", ""),
        "published": result.get("published", ""),
        "thumbnail": result.get("thumbnail", {}).get("src", "") if
result.get("thumbnail") else ""
    }
    search_results.append(search_result)

# Save results to file for reference
try:
    results_file = results_dir / f"search_{query.replace(' ', '_')}
[:50]}.json"
    with open(results_file, 'w', encoding='utf-8') as f:
        json.dump({
            "query": query,
            "timestamp": data.get("query", {}).get("posted_at", ""),
            "results": search_results
        }, f, indent=2, ensure_ascii=False)
    logger.info(f"Search results saved to {results_file}")
except Exception as e:
    logger.warning(f"Failed to save search results: {e}")

logger.info(f"Found {len(search_results)} search results")

return {
    "success": True,
    "query": query,
    "total_results": len(search_results),
    "results": search_results
}

except Exception as e:
    logger.exception("Search operation failed")
    return {
        "success": False,
        "error": str(e),
        "results": []
    }

@mcp.tool(name="search_news", description="Search for news using Brave
Search API")
def search_news(query: str, count: int = 5) -> Dict[str, Any]:
    """
    Search for news using Brave Search API

    Args:
        query: Search query string
        count: Number of news results to return (max 20)

    Returns:
        Dictionary containing news search results
    """
    try:
        logger.info(f"Searching news for: {query}")

        if not api_key:

```

```

        return {
            "success": False,
            "error": "BRAVE_API_KEY not configured",
            "results": []
        }

# Limit count to reasonable range
count = max(1, min(count, 20))

# Request headers
headers = {
    "Accept": "application/json",
    "Accept-Encoding": "gzip",
    "X-Subscription-Token": api_key
}

# Request parameters for news search
params = {
    "q": query,
    "count": count,
    "search_lang": "en",
    "country": "US",
    "safesearch": "moderate",
    "freshness": "pd", # Past day for latest news
    "text_decorations": False,
    "result_filter": "news" # Focus on news results
}

# Make API request
try:
    response = requests.get(
        BRAVE_SEARCH_URL,
        headers=headers,
        params=params,
        timeout=30
    )
    response.raise_for_status()
except requests.exceptions.RequestException as e:
    logger.error(f"News search API request failed: {e}")
    return {
        "success": False,
        "error": f"News search API request failed: {str(e)}",
        "results": []
    }

# Parse response
try:
    data = response.json()
except json.JSONDecodeError as e:
    logger.error(f"Failed to parse news search response: {e}")
    return {
        "success": False,
        "error": "Failed to parse news search response",
        "results": []
    }

```

```

# Extract news results
news_results = []

# Check for news section in response
news_data = data.get("news", {}).get("results", [])
if not news_data:
    # Fallback to web results if no dedicated news section
    news_data = data.get("web", {}).get("results", [])

for result in news_data:
    news_result = {
        "title": result.get("title", ""),
        "url": result.get("url", ""),
        "description": result.get("description", ""),
        "published": result.get("age", result.get("published", "")),
        "source": result.get("profile", {}).get("name", "") if
result.get("profile") else "",
        "thumbnail": result.get("thumbnail", {}).get("src", "") if
result.get("thumbnail") else ""
    }
    news_results.append(news_result)

logger.info(f"Found {len(news_results)} news results")

return {
    "success": True,
    "query": query,
    "total_results": len(news_results),
    "results": news_results
}

except Exception as e:
    logger.exception("News search operation failed")
    return {
        "success": False,
        "error": str(e),
        "results": []
    }

if __name__ == "__main__":
    logger.info("Starting Brave Search MCP Server")
    try:
        mcp.run(transport="stdio")
    except Exception as e:
        logger.exception("Search server crashed")
        # Add pause to see error in Windows
        input("Press Enter to exit...")
        raise

```

Agent — main.py


```

from crewai import Agent, Task, Crew, LLM
from crewai_tools import MCPServerAdapter
from mcp import StdioServerParameters
import sys
import platform
from pathlib import Path
import os
import warnings
from pydantic import PydanticDeprecatedSince20
from dotenv import load_dotenv
import traceback
import subprocess
from pydantic import BaseModel, Field
class Summary(BaseModel):
    summary: str = Field(description="A detailed summary of the research findings")
    image_path: str = Field(description="The path to the image file created by the agent")

# Load environment variables
load_dotenv()

def get_available_llm():
    """Get the first available LLM from environment variables"""
    llm_configs = [
        {
            "name": "Groq Llama",
            "model": "groq/llama-3.3-70b-versatile",
            "api_key_env": "GROQ_API_KEY",
            "temperature": 0.7
        },
        {
            "name": "OpenAI GPT-4",
            "model": "gpt-4o-mini",
            "api_key_env": "OPENAI_API_KEY",
            "temperature": 0.7
        },
        {
            "name": "Anthropic Claude",
            "model": "claude-3-haiku-20240307",
            "api_key_env": "ANTHROPIC_API_KEY",
            "temperature": 0.7
        },
        {
            "name": "Ollama Local",
            "model": "ollama/llama3.2",
            "api_key_env": None, # No API key needed for local
            "temperature": 0.7
        }
    ]

    print("🔍 Checking available LLM providers...")

    for config in llm_configs:
        try:
            if config["api_key_env"] is None:

```

```

        # For local models like Ollama, try without API key
        print(f"⚡ Trying {config['name']} (Local)...")
        llm = LLM(
            model=config["model"],
            temperature=config["temperature"],
            max_tokens=1000,
        )
        print(f"✅ Using {config['name']}: {config['model']}")
        return llm
    else:
        api_key = os.getenv(config["api_key_env"])
        if api_key:
            print(f"⚡ Trying {config['name']}...")
            llm = LLM(
                model=config["model"],
                temperature=config["temperature"],
                api_key=api_key
            )
            print(f"✅ Using {config['name']}: {config['model']}")
            return llm
        else:
            print(f"⚠️ {config['name']} API key not found in
environment")
    except Exception as e:
        print(f"❌ {config['name']} failed: {str(e)[:100]}...")
        continue

    # Fallback to basic configuration if all else fails
    print(f"⚠️ Using fallback LLM configuration...")
    return LLM(
        model="groq/llama-3.3-70b-versatile",
        temperature=0.7,
        api_key=os.getenv("GROQ_API_KEY", "")
    )

# Configure LLM with fallback options
llm = get_available_llm()

# Suppress warnings
warnings.filterwarnings("ignore", category=PydanticDeprecatedSince20)

# Get current directory
base_dir = Path(__file__).parent.resolve()

print(f"Python executable: {sys.executable}")
print(f"Current directory: {os.getcwd()}")
print(f"Base directory: {base_dir}")

# Determine correct npx command for Windows
npx_cmd = "npx.cmd" if platform.system() == "Windows" else "npx"

def check_npx_availability():
    """Check if npx is available and working"""
    try:
        result = subprocess.run([npx_cmd, "--version"],
                                capture_output=True, text=True, timeout=10)

```

```

        if result.returncode == 0:
            print(f"✓ NPX is available: {result.stdout.strip()}")
            return True
        else:
            print(f"✗ NPX check failed: {result.stderr}")
            return False
    except Exception as e:
        print(f"✗ NPX not available: {e}")
        return False

def check_python_server():
    """Check if the Python image server exists"""
    server_path = base_dir / "servers" / "image_server.py"
    if server_path.exists():
        print(f"✓ Python image server found: {server_path}")
        return True
    else:
        print(f"✗ Python image server not found: {server_path}")
        return False

def check_search_server():
    """Check if the Python search server exists"""
    server_path = base_dir / "servers" / "search_server.py"
    if server_path.exists():
        print(f"✓ Python search server found: {server_path}")
        return True
    else:
        print(f"✗ Python search server not found: {server_path}")
        return False

def get_working_servers():
    """Get list of working server configurations"""
    working_servers = []

    print("\n" + "="*50)
    print("DIAGNOSING MCP SERVERS")
    print("="*50)

    # Check Python image server first (most likely to work)
    python_server_available = check_python_server()
    if python_server_available:
        image_server_params = StdioServerParameters(
            command="python",
            args=[
                str(base_dir / "servers" / "image_server.py"),
            ],
            env={"UV_PYTHON": "3.12", **os.environ},
        )
        working_servers.append(("Image Server", image_server_params))
        print("✓ Image server configured")
    else:
        print("✗ Skipping Image server (server file not found)")

    # Check Python search server
    search_server_available = check_search_server()
    if search_server_available:

```

```

        search_server_params = StdioServerParameters(
            command="python",
            args=[
                str(base_dir / "servers" / "search_server.py"),
            ],
            env={"UV_PYTHON": "3.12", **os.environ},
        )
        working_servers.append(("Python Search Server",
search_server_params))
        print("✓ Python search server configured")
    else:
        print("✗ Skipping Python search server (server file not found)")

# Check NPX availability for filesystem server only
npx_available = check_npx_availability()

# Only add NPX servers if Node.js version is recent enough
if npx_available:
    node_version_check = check_node_version()
    if node_version_check:
        # Filesystem server configuration
        filesystem_server_params = StdioServerParameters(
            command=npx_cmd,
            args=[
                "-y",
                "@modelcontextprotocol/server-filesystem",
                os.path.join(os.path.expanduser("~"), "Downloads")
            ],
        )
        working_servers.append(("Filesystem Server",
filesystem_server_params))
        print("✓ Filesystem server configured")
    else:
        print("⚠ Skipping NPX filesystem server due to Node.js version
compatibility issues")
        print("💡 To enable filesystem server, update Node.js to version
18+ or 20+")
        print("    Visit: https://nodejs.org/en/download/")
    else:
        print("✗ Skipping NPX filesystem server (NPX not available)")

print(f"\nFound {len(working_servers)} server configurations")
return working_servers

def check_node_version():
    """Check if Node.js version is compatible"""
    try:
        result = subprocess.run(["node", "--version"],
                                capture_output=True, text=True, timeout=10)
        if result.returncode == 0:
            version = result.stdout.strip()
            print(f"Node.js version: {version}")
            # Extract major version number
            major_version = int(version.lstrip('v').split('.')[0])
            if major_version >= 18:
                print("✓ Node.js version is compatible")

```

```

        return True
    else:
        print(f"⚠️ Node.js version {version} may be too old
(recommend v18+)")
        return False
    return False
except Exception as e:
    print(f"✖ Cannot check Node.js version: {e}")
    return False

class CustomMCPServerAdapter(MCPServerAdapter):
    """Custom MCP Server Adapter with increased timeout"""
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.timeout = 90 # Increase timeout to 90 seconds

def test_servers_individually(server_configs):
    """Test each server individually to identify problematic ones"""
    working_servers = []

    print("\n" + "="*50)
    print("TESTING SERVERS INDIVIDUALLY")
    print("="*50)

    for name, server_params in server_configs:
        print(f"\nTesting {name}...")
        try:
            with CustomMCPServerAdapter([server_params]) as tools:
                print(f"✓ {name} connected successfully!")
                print(f" Available tools: {[tool.name for tool in tools]}")
                working_servers.append(server_params)
        except Exception as e:
            print(f"✖ {name} failed: {str(e)[:100]}...")
            continue

    return working_servers

def create_agent_and_tasks(tools=None):
    """Create agent and tasks with or without tools"""
    tools_list = tools or []

    # Adjust role and tasks based on available tools
    if tools_list:
        tool_names = [getattr(tool, 'name', 'unknown') for tool in
tools_list]
        print(f"Agent will have access to: {tool_names}")

        role = "AI Research Creator with Tools"
        goal = "Research topics thoroughly using available MCP tools, create
comprehensive diagrams, and save summaries"
        backstory = "An AI researcher and creator that specializes in using
MCP tools to gather information, create visual representations, and save
findings."
    else:
        role = "AI Research Creator"
        goal = "Research topics using built-in knowledge and create

```

```

comprehensive analysis"
    backstory = "An AI researcher that specializes in analyzing topics
and providing detailed insights using available knowledge."

    agent = Agent(
        role=role,
        goal=goal,
        backstory=backstory,
        tools=tools_list,
        llm=llm,
        verbose=True,
    )

    if tools_list:
        research_task = Task(
            description="Research the topic '{topic}' thoroughly using
available MCP tools. If image creation tools are available, create an in-
depth diagram showing how the topic works, including key components,
processes, and relationships.",
            expected_output="A comprehensive research summary and, if
possible, a successfully created diagram/image illustrating the topic.",
            agent=agent,
        )

        summary_task = Task(
            description="Create a detailed summary of your research
findings. If filesystem tools are available, save it as a text file in the
Downloads folder. Include key insights, important details, and references to
any diagrams created.",
            expected_output="A detailed summary of research findings,
preferably saved as a text file if filesystem access is available. The final
response should be in the format of a pydantic model Summary",
            agent=agent,
            output_pydantic=Summary
        )
    else:
        research_task = Task(
            description="Research and analyze the topic '{topic}' thoroughly
using your knowledge. Provide detailed insights about how it works,
including key components, processes, and relationships.",
            expected_output="A comprehensive analysis and explanation of the
topic with detailed insights.",
            agent=agent,
        )

        summary_task = Task(
            description="Create a detailed summary of your analysis,
highlighting the most important aspects, key insights, and practical
implications of the topic.",
            expected_output="A well-structured summary with key findings and
insights about the topic. The final response should be in the format of a
pydantic model Summary",
            agent=agent,
            output_pydantic=Summary,
            markdown=True, # Enable markdown formatting for the final
output

```

```

        output_file="report.md"
    )

    return agent, [research_task, summary_task]

def main():
    """Main function to run the CrewAI application"""
    # Get available server configurations
    server_configs = get_working_servers()

    if not server_configs:
        print("\n⚠️ No MCP servers available. Running in fallback mode only.")
        run_fallback_mode()
        return

    # Test servers individually to find working ones
    working_server_params = test_servers_individually(server_configs)

    if not working_server_params:
        print("\n⚠️ No MCP servers are working. Running in fallback mode.")
        run_fallback_mode()
        return

    try:
        print(f"\n✓ Using {len(working_server_params)} working MCP server(s)")
        print("Initializing MCP Server Adapter...")

        with CustomMCPDataAdapter(working_server_params) as tools:
            print("Successfully connected to MCP servers!")
            print(f"Available tools: {[tool.name for tool in tools]}")

            # Create agent and tasks with MCP tools
            agent, tasks = create_agent_and_tasks(tools)

            # Create crew with error handling
            crew = Crew(
                agents=[agent],
                tasks=tasks,
                verbose=True,
                reasoning=True,
            )

            # Get user input
            topic = input("\nPlease provide a topic to research: ").strip()
            if not topic:
                topic = "artificial intelligence"
                print(f"No topic provided, using default: {topic}")

            # Execute crew with retry mechanism
            max_retries = 2
            for attempt in range(max_retries + 1):
                try:
                    print(f"\nStarting research on: {topic} (Attempt {attempt + 1})")

```

```

        result = crew.kickoff(inputs={"topic": topic})
        # print("\n" + "="*50)
        # print("FINAL RESULT FROM THE AGENT")
        # print("="*50)

        response = result["summary"]
        print(response)
        print(f"Summary task output :{tasks[1].output}")
        return response
    except Exception as e:
        if attempt < max_retries:
            print(f"⚠️ Attempt {attempt + 1} failed: {str(e)}

[:100]}...)

            print(f"🔄 Retrying... ({attempt + 2}/{max_retries +
1}))")

            continue
        else:
            print(f"❌ All attempts failed. Error: {e}")
            raise e

except Exception as e:
    print(f"Error running with MCP tools: {e}")
    traceback.print_exc()
    print("\nFalling back to basic agent without MCP tools...")
    run_fallback_mode()

def run_fallback_mode():
    """Run the application without MCP tools"""
    print("\n" + "="*50)
    print("RUNNING IN FALLBACK MODE")
    print("="*50)

    # Create fallback agent without MCP tools but with LLM
    agent, tasks = create_agent_and_tasks()

    crew = Crew(
        agents=[agent],
        tasks=tasks,
        verbose=True,
        reasoning=True,
    )

    # Get user input for fallback
    topic = input("Please provide a topic to research (fallback mode):
").strip()
    if not topic:
        topic = "artificial intelligence"
        print(f"No topic provided, using default: {topic}")

    print(f"\nStarting research on: {topic} (without MCP tools)")
    result = crew.kickoff(inputs={"topic": topic})
    print("\n" + "="*50)
    print("FINAL RESULT (Fallback Mode):")
    print("="*50)
    print(result["summary"])
    return result["summary"]

```



```
if __name__ == "__main__":
    print("🚀 Starting CrewAI MCP Demo")
    print("\n📖 Setup Instructions:")
    print("    For more MCP servers, update Node.js to v18+:  
https://nodejs.org")
    print("    Add API keys to .env file for additional LLM providers")
    print("    Supported: GROQ_API_KEY, OPENAI_API_KEY, ANTHROPIC_API_KEY,  
BRAVE_API_KEY")
    result = main()
    #print(result)
```

User Interface using Streamlit-app.py

```

import streamlit as st
import subprocess
import sys
import os
from pathlib import Path
import glob
from PIL import Image
import re

def find_venv_python():
    """Find the correct Python executable from virtual environment"""
    current_dir = Path(__file__).parent
    possible_venv_paths = [
        os.path.join(current_dir, ".venv", "Scripts", "python.exe"),
        os.path.join(current_dir, "venv", "Scripts", "python.exe"),
        os.path.join(current_dir, ".venv", "bin", "python"),
        os.path.join(current_dir, "venv", "bin", "python"),
    ]

    for path in possible_venv_paths:
        if os.path.exists(path):
            return path
    return sys.executable

def run_research(topic):
    """Run main.py with the given topic and return the result"""
    current_dir = Path(__file__).parent
    python_executable = find_venv_python()

    # Prepare environment with UTF-8 encoding
    env = os.environ.copy()
    env['PYTHONIOENCODING'] = 'utf-8'
    env['PYTHONLEGACYWINDOWSSTDIO'] = '1'

    try:
        # Run main.py as subprocess
        process = subprocess.Popen(
            [python_executable, "main.py"],
            cwd=current_dir,
            stdin=subprocess.PIPE,
            stdout=subprocess.PIPE,
            stderr=subprocess.PIPE,
            text=True,
            encoding='utf-8',
            errors='replace',
            env=env
        )

        # Send topic as input
        stdout, stderr = process.communicate(input=topic + "\n",
        timeout=300)

        if process.returncode == 0:
            # Extract final result from stdout
            return extract_final_result(stdout), None
        else:

```



```

"""Get the latest images from the images folder"""
images_dir = Path("servers/images")
if not images_dir.exists():
    return []

# Get all image files
image_extensions = ['*.jpg', '*.jpeg', '*.png', '*.gif', '*.bmp']
image_files = []

for ext in image_extensions:
    image_files.extend(glob.glob(str(images_dir / ext)))

if not image_files:
    return []

# Sort by modification time (newest first)
image_files.sort(key=os.path.getmtime, reverse=True)

# Return top 5 most recent images
return image_files[:1]

def main():
    st.set_page_config(
        page_title="CrewAI-MCP Research Assistant",
        page_icon="🔍",
        layout="wide"
    )

    st.title("🔍 CrewAI-MCP Study Assistant")
    st.markdown("Enter a topic to research and generate comprehensive insights with visual diagrams.")

    # Topic input
    topic = st.text_input(
        "Research Topic:",
        placeholder="e.g., Explain photosynthesis process, Machine learning algorithms, etc.",
        help="Enter any topic you want to research in detail"
    )

    # Research button
    if st.button("🚀 Start Research", type="primary", disabled=not topic.strip()):
        if topic.strip():
            with st.spinner(f"🔍 Researching '{topic}'... This may take a few minutes."):
                result, error = run_research(topic.strip())
                print(f"Result from CREWAI : {result}")

            if result:
                st.success("✅ Research completed successfully!")
                print(f"Result from CREWAI : {result}")
                # Store results in session state
                st.session_state['research_result'] = result
                st.session_state['research_topic'] = topic.strip()
                st.session_state['latest_images'] = get_latest_images()

```

```

        else:
            st.error(f"❌ Research failed: {error}")

# Display results and images side by side
if 'research_result' in st.session_state:
    # Create a divider
    st.divider()
    st.subheader(f"Research Results:
{st.session_state.get('research_topic', 'Unknown Topic')}")

    # Create two columns for side-by-side display
    col1, col2 = st.columns([2, 1]) # Results get 2/3 width, Images get
1/3 width

    # Left column - Research Results
    with col1:
        st.markdown("### 📄 Summary Results")

        # Display the result in markdown format
        result_text = st.session_state['research_result']
        pattern = re.compile(r'\x1b\[\d;]*m')
        result_text = pattern.sub('', result_text)

        # Create a scrollable container for long content
        with st.container():
            st.markdown(result_text)

        # Add download button for the result
        st.download_button(
            label="📄 Download Results as Text",
            data=result_text,
            file_name=f"research_{st.session_state.get('research_topic',
'topic').replace(' ', '_')}.txt",
            mime="text/plain"
        )

    # Right column - Generated Images
    with col2:
        st.markdown("### 🖼️ Generated Images")

        images = st.session_state.get('latest_images', [])

        if images:
            st.success(f"Found {len(images)} image(s)")

            # Display images vertically stacked
            for idx, image_path in enumerate(images):
                try:
                    # Open and display image
                    img = Image.open(image_path)

                    st.image(
                        img,
                        caption=f"Generated: {Path(image_path).name}",
                        use_container_width=True
                    )

```

```

        # Add download button for each image
        with open(image_path, "rb") as file:
            st.download_button(
                label=f"⬇️ Download",
                data=file.read(),
                file_name=Path(image_path).name,
                mime="image/jpeg",
                key=f"download_img_{idx}"
            )

        # Add spacing between images if there are multiple
        if idx < len(images) - 1:
            st.markdown("---")

    except Exception as e:
        st.error(f"Error loading image: {str(e)}")
    else:
        st.info("🖼️ Images will appear here after research completion.")

    with st.expander("ℹ️ About Images"):
        st.markdown("""
            **How it works:**
            - Images are automatically generated during research
            - Saved to `servers/images/` folder
            - Displayed here sorted by creation time
            - Download button available for each image
            """)

if __name__ == "__main__":
    main()

```

🎯 Results & Performance

What You Get:

- ⚡ **Lightning-fast research:** 30–60 seconds for comprehensive studies
- 🎨 **Visual enhancement:** AI-generated images for every topic
- 📊 **Structured output:** Organized, downloadable results
- 🔍 **Deep insights:** Multi-source information synthesis
- 🌐 **User-friendly:** Intuitive web interface

CrewAI-MCP Study Assistant

Enter a topic to research and generate comprehensive insights with visual diagrams.

Research Topic:

Describe the benefits of consuming Pomegranate

Start Research

Research completed successfully!

Research Results: Describe the benefits of consuming Pomegranate

Summary Results

The benefits of consuming pomegranate include reducing the risk of cardiovascular diseases, fighting inflammation, protecting cells from damage, and slowing the aging process. Pomegranate juice has been associated with a significant reduction in risk factors for cardiovascular diseases, including improvements in blood pressure and lipid profiles. It is also rich in antioxidants like punicalagins and anthocyanins, which help protect cells from damage and slow the aging process. Here is a diagram showing the benefits of consuming pomegranate: The diagram shows the different components and processes involved in the benefits of consuming pomegranate. It includes the following elements:

- Pomegranate fruit: This is the main source of the benefits, and it is rich in antioxidants and other nutrients.
- Antioxidants: These are the compounds that help protect cells from damage and slow the aging process. They include punicalagins and anthocyanins.
- Inflammation reduction: The antioxidants in pomegranate help reduce inflammation, which can reduce the risk of chronic diseases.
- Cell protection: The antioxidants also help protect cells from damage, which can help slow the aging process.
- Heart health: The benefits of consuming pomegranate include reducing the risk of cardiovascular diseases, including improvements in blood

Generated Images

Found 1 image(s)



Describe lifestyle of neanthendral man

Start Research

Research completed successfully!

Research Results: Describe lifestyle of neanthendral man

Summary Results

The Neanderthal man's lifestyle was likely centered around hunting and gathering, with a focus on survival and community. They likely lived in small groups, with a mix of males and females, and worked together to hunt and gather food. They also likely had a strong sense of tool use and creation, with evidence of complex stone tools and potentially even early forms of art. Here is a diagram illustrating the lifestyle of Neanderthal man: Neanderthal man hunting and gathering, with a focus on survival and community, using complex stone tools and potentially even early forms of art. The key components of their lifestyle include:

- Hunting: Neanderthals likely hunted large game such as mammoths and bison, using coordinated attacks and complex strategies.
- Gathering: They also gathered fruits, nuts, and other plant materials, potentially using tools to aid in the process.

Download Results as Text

Generated Images

Found 1 image(s)



✓ Research completed successfully!

Research Results: Describe Fusion process

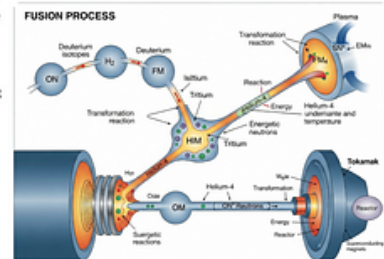
Summary Results

The fusion process is a complex phenomenon that involves the combination of two or more atomic nuclei to form a single, heavier nucleus, releasing energy in the process. This process has the potential to generate electricity and has been proposed as a form of power generation. The key components of the fusion process include the atomic nuclei, the plasma state, and the reaction chamber. The process involves the ionization of the atomic nuclei, the formation of a plasma, and the acceleration of the plasma to high temperatures and pressures, resulting in the fusion reaction. The fusion process can be illustrated using a diagram, which shows the key components and processes involved. The diagram depicts the atomic nuclei, the plasma state, and the reaction chamber, as well as the ionization, acceleration, and fusion reaction stages. In conclusion, the fusion process is a complex and highly energetic phenomenon that has the potential to generate electricity and provide a new source of power. Further research and development are needed to fully understand and harness the power of fusion. Diagram: A diagram of the fusion process, including key components and processes, has been created using the image_creation_openai tool and is available at the specified image path. The diagram illustrates the atomic nuclei, plasma state, reaction chamber, ionization, acceleration, and fusion reaction stages, providing a visual representation of the fusion process.

Download Results as Text

Generated Images

Found 1 image(s)



Download

Performance Metrics:

- **Search Speed:** ~5–10 seconds for web results
- **Image Generation:** ~15–30 seconds per image
- **Summary Creation:** ~20–40 seconds for comprehensive analysis
- **Overall Workflow:** ~60–120 seconds total

Future Scope: The Next Evolution

Immediate Enhancements

- **PDF Analysis:** Upload and analyze documents
- **Video Content:** YouTube video summarization
- **Voice Interface:** Speech-to-text research queries
- **Mobile App:** Native iOS/Android applications

Advanced Features

- **Knowledge Graphs:** Visual relationship mapping
- **Data Visualization:** Interactive charts and graphs
- **Citation Management:** Academic reference generation
- **Collaborative Research:** Multi-user workspace

Technical Improvements

- ⚡ **RAG Integration:** Vector database for better context
- 🔗 **Real-time Updates:** Live research monitoring
- 🎯 **Personalization:** User-specific preferences
- 🛡️ **Enterprise Security:** Advanced authentication

Ecosystem Expansion

- 🔌 **Plugin Architecture:** Third-party integrations
- 📊 **Analytics Dashboard:** Usage insights and metrics
- 🎓 **Educational Tools:** Flashcards, quizzes, mind maps
- 🏢 **Enterprise Version:** Team collaboration features

References & Resources

Essential Documentation

- **CrewAI Framework:** <https://docs.crewai.com>
- **Model Context Protocol:** <https://modelcontextprotocol.io>
- **Streamlit Documentation:** <https://docs.streamlit.io>
- **Brave Search API:** <https://api.search.brave.com/app/documentation>

Technical Resources

Conclusion: Your AI Research Revolution Starts Now

We've just built something extraordinary — a **complete AI-powered research ecosystem** that showcases the future of intelligent applications. This isn't just about code; it's about **transforming how we interact with information**.

What We've Accomplished:

- ✅ **Seamless Integration:** MCP protocol connecting AI agents with real-world tools
- ✅ **Beautiful Interface:** Modern, responsive web application
- ✅ **Practical Value:** Real research capabilities with downloadable results

-  **Scalable Architecture:** Foundation for enterprise-grade applications
-  **Future-Ready:** Built with cutting-edge technologies

The Bigger Picture:

This project demonstrates the **convergence of multiple AI technologies**:

- Multi-agent systems working in harmony
- Protocol-driven tool integration
- User-centric interface design
- Real-world API utilization

Your Next Steps:

1.  **Build It:** Follow our implementation guide
2.  **Customize It:** Add your own features and styling
3.  **Scale It:** Deploy for your team or organization
4.  **Share It:** Contribute to the open-source community

Final Thoughts:

The **Model Context Protocol** represents a paradigm shift in AI application development. By standardizing how AI agents interact with external tools, MCP opens doors to **unlimited possibilities**.

Your Study Assistant is just the beginning. With this foundation, you can build:

-  **Enterprise research platforms**
-  **Educational AI tutors**
-  **Business intelligence dashboards**
-  **Scientific research tools**

The future of AI is collaborative, contextual, and user-centric — and you're now equipped to build it.

Ready to revolutionize research? Start building your MCP-powered Study Assistant today and join the next wave of AI innovation! 

Happy Building!  